

A Materialised Three-Way Merge for AegisSheet

Exploratory research note

FP Launchpad

2 September 2026

Abstract

The verified AegisSheet model stores the whole event history and merges by union; each event carries the timestamp set of its causal past, so state is quadratic in history length. This note asks whether the ancestor argument of a three-way merge can replace that metadata. We define a materialised state with a componentwise three-way merge and test it differentially against the existing model on random version DAGs. No counterexample appears in 3000 executions when a removal names the tokens it kills; clearing tokens instead converges but breaks commutation. Three hand-derived witnesses, checked by the executable reference, show that the last position of a removed row or column cannot be dropped while a concurrent revival is possible, so the retention residue is one register per axis identifier until removal is causally stable. The measured state is linear rather than quadratic in history. Purge needs a mask of cutoff and coordinates per marker, because the current model deliberately masks writes concurrent with the marker and below its cutoff, but it needs no acknowledgement protocol: dead identifiers can be retired immediately once the register is kept. None of the new claims is machine-checked; the mechanization target is a conditioned Join for the token and version components composed with the register and the mask, plus an observational-equivalence theorem against the union model.

1 Goal, hypotheses, and status

Goal. Re-encode AegisSheet as a genuine three-way MRDT with a materialised state so that the lowest-common-ancestor argument of `merge` replaces the per-event causal set, the append-only event log, and, where possible, the roster-acknowledged purge protocol, while preserving every observation of the current model.

Hypotheses and falsifiers.

- H1** Cells and axes forget. A state of live keep tokens, latest-position registers, active cell versions, and active range versions, merged componentwise by the rule $\text{mvr}(l, a, b) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$ for sets and last-writer-wins for registers, yields the same observation as the union model at every reachable version. Falsifier: a version DAG on which the observations differ, two merge topologies over one event set that disagree, or a merge that differs from replaying the union.
- H2** The latest position of a removed axis identifier can be dropped once no live range names it. Falsifier: two histories with identical live sheets whose continuations require different positions for the identifier.

H3 Purge dissolves into ordinary version deletion once state is materialised. Falsifier: a late concurrent write that revives content the paper requires to stay purged.

Status. Table 1 uses the evidence vocabulary conjectured, refuted, machine-checked, validated, measured, specified, and staged. None of the candidate claims in this note is machine-checked; the reference definitions and the baseline theorems cited from the repository are. The Python reference is validated against the Lean model only on the fixtures listed in Section 9.

Claim	Status	Evidence
H1, named removal	validated on honest executions; conjectured in general	no counterexample in 3000 executions with Lamport clocks, laggard replicas, and purge markers: 110,560 versions, 19,240 registered-ancestor and 461 virtual-base merges, three seeds; all 90,859 generated events pass a transcribed <code>applicableB</code> after erasing <i>kills</i> ; Lean proof owed
H1, clearing removal	validated for observation; refuted for replay	converges and matches the reference; 675 of 3000 executions have a merge that differs from timestamp-order replay
H2	refuted	three hand-derived witnesses checked by the reference (Section 5); 35 to 41 failing executions per 1000 for the register-dropping variant
H3, strict	refuted	purge as plain deletion diverges in 6, 3, 3 executions per 1000, always on a write concurrent with a marker and below its cutoff
H3, with mask	validated	a mask of cutoff and coordinates per marker reproduces the reference; 0 failing executions
Retirement without acknowledgements	validated	dropping a dead identifier’s tokens and <code>known</code> entry immediately, or with either evidence rule, has 0 failing executions; dropping the register fails
State growth	measured	Table 3: reference quadratic, materialised linear
Undo revives a later removal	finding, reproduced; excluded by decision D1	Section 6; legacy behaviour of the current model

Table 1: Status of each claim.

Trusted boundary. The reference semantics is the executable Lean model in `Sal/MRDTs/Instances/AegisSheet.lean`, which the repository validates against Tables 3 and 4 of the AegisSheet paper. The Python transcription of that model is trusted only to the extent of the fixture check. The candidate design is specified here and implemented in Python; it has no Lean declaration.

2 The reference model

The displays below transcribe the Lean definitions named in each definition’s title. The purge marker’s fields and its masking clause are stated in Section 7 rather than displayed here (labelled compression).

Definition 2.1 (events and state; `SheetOp`, `D`). An event is $e = (t, r, \langle \text{seen}, \text{cmd} \rangle)$ with Lamport timestamp t , replica r , a finite set `seen` of timestamps, and a command that is either a direct action or `undo`(t' , inv) carrying an inverse action. The effect `action`(e) is the direct action or inv . Actions are

$\text{axis}(\text{kind}, \alpha, \text{id}, \text{before}, \text{after}) \quad \alpha \in \{\text{row}, \text{col}\}, \text{kind} \in \{\text{insert}, \text{move}, \text{remove}, \text{restore}\},$
 $\text{cell}(\rho, \gamma, \text{before}, \text{after}, \text{overwrites}) \quad \text{before}, \text{after} \text{ finite value sets, overwrites timestamps,}$
 $\text{range}(\text{id}, \text{before}, \text{after}, \text{overwrites}) \quad \text{before}, \text{after} \in \text{Option}(\text{RangeSpec}).$

The MRDT state is a finite set E of events; $\text{do}(E, e) = E \cup \{e\}$; $\text{merge}(_, a, b) = a \cup b$. The ancestor argument is unused.

Definition 2.2 (axis liveness; `keepsAxis`, `removesAxis`, `axisLive`). An event e keeps (α, id) when it is an axis update of that identifier with $\text{after} \neq \perp$, or a cell update whose α -coordinate is id . It removes (α, id) when it is an axis update of that identifier with $\text{after} = \perp$.

$\text{keepTimes}_E(\alpha, \text{id}) = \{t(e) \mid e \in E, e \text{ keeps } (\alpha, \text{id})\},$
 $\text{removed}_E(\alpha, \text{id}, \tau) \iff \exists e \in E. e \text{ removes } (\alpha, \text{id}) \wedge \tau \in \text{seen}(e),$
 $\text{live}_E(\alpha, \text{id}) \iff \text{known}_E(\alpha, \text{id}) \wedge \exists \tau \in \text{keepTimes}_E(\alpha, \text{id}). \neg \text{removed}_E(\alpha, \text{id}, \tau),$

where known_E holds when some axis update of the identifier is in E . A removal defeats exactly the keep tokens it observed; a keep it did not observe survives it.

Definition 2.3 (positions, cells, ranges; `axisPositions`, `cellValues`, `rangeValues`). A candidate for (α, id) is an axis update with $\text{after} \neq \perp$; $\text{positions}_E(\alpha, \text{id})$ is the set of after values of candidates with no later candidate, a singleton or empty. A cell version e at (ρ, γ) is overwritten when some cell update in E lists $t(e)$ in its overwrites; $\text{cells}_E(\rho, \gamma)$ is the union of after over non-overwritten versions when both axes are live, and $\emptyset$ otherwise. Range versions are analogous, and $\text{ranges}_E(\text{id})$ collects their non- $\perp$ after specifications.

Definition 2.4 (range resolution; `resolveRange`). For a specification $(\rho_1, \rho_2, \gamma_1, \gamma_2)$, a live endpoint resolves to itself. A dead first endpoint resolves to the live identifier at the least live position greater than the endpoint's last position; a dead last endpoint to the live identifier at the greatest live position less than it. Resolution fails when an endpoint has no position or the resolved rectangle is inverted.

Definition 2.5 (issuance; `applicableB`, `validUndo`, `inverseFor`). An event is issuable at E when $t(e) \notin \text{times}(E)$, $\text{seen}(e) = \text{times}(E)$, $t(e)$ exceeds every timestamp in E , and its before-image and overwrites list equal the current values at E (compressed: the per-kind clauses of `directApplicable`). An undo is issuable when its target is in $\text{seen}(e)$, was issued by the same replica, and the carried inverse equals `inverseFor` of the target, which swaps before and after and sets $\text{overwrites} = \{t(\text{target})\}$ for cells and ranges. No clause requires the axes of an undone cell write to be live.

The clause $\text{seen}(e) = \text{times}(E)$ is the cost this note attacks: every event stores the timestamp set of its entire causal past.

3 The candidate design

Definition 3.1 (materialised state).

$\text{known} : \{\text{row}, \text{col}\} \rightarrow \mathcal{P}(\text{Id})$	<i>grow-only identifier sets,</i>
$\text{tok} : (\{\text{row}, \text{col}\} \times \text{Id}) \rightarrow \mathcal{P}(\text{Ts})$	<i>live keep tokens,</i>
$\text{pos} : (\{\text{row}, \text{col}\} \times \text{Id}) \rightarrow \text{Ts} \times \text{Position}$	<i>latest position register,</i>
$\text{cells} : (\text{Id} \times \text{Id}) \rightarrow \mathcal{P}(\text{Ts} \times \mathcal{P}(\text{Value}))$	<i>active cell versions,</i>
$\text{ranges} : \text{Id} \rightarrow \mathcal{P}(\text{Ts} \times \text{Option}(\text{RangeSpec}))$	<i>active range versions.</i>

A removal carries kills, the live tokens of its identifier at issuance (the named variant); the clearing variant carries nothing and empties the token set.

Definition 3.2 (update).

$$\begin{aligned}
\text{axis}(_, \alpha, id, _, p \neq \perp) &: \text{tok}(\alpha, id) += \{t\}, \quad \text{pos}(\alpha, id) := \max_{\text{ts}}(\text{pos}(\alpha, id), (t, p)), \quad \text{known} += id; \\
\text{axis}(_, \alpha, id, _, \perp) &: \text{tok}(\alpha, id) -= \text{kills} \quad (\text{clearing: } \text{tok}(\alpha, id) := \emptyset); \\
\text{cell}(\rho, \gamma, _, v, ov) &: \text{tok}(\text{row}, \rho) += \{t\}, \quad \text{tok}(\text{col}, \gamma) += \{t\}, \\
&\quad \text{cells}(\rho, \gamma) := \{x \in \text{cells}(\rho, \gamma) \mid \text{ts}(x) \notin ov\} \cup \{(t, v)\}; \\
\text{range}(id, _, s, ov) &: \text{ranges}(id) := \{x \in \text{ranges}(id) \mid \text{ts}(x) \notin ov\} \\
&\quad \cup \{(t, s)\}.
\end{aligned}$$

Definition 3.3 (merge). For $\text{mvr}(l, a, b) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$,

$$\begin{aligned}
\text{merge}(l, a, b).\text{known} &= l.\text{known} \cup a.\text{known} \cup b.\text{known}, \\
\text{merge}(l, a, b).\text{tok}(k) &= \text{mvr}(l.\text{tok}(k), a.\text{tok}(k), b.\text{tok}(k)), \\
\text{merge}(l, a, b).\text{pos}(k) &= \max_{\text{ts}}\{x.\text{pos}(k) \mid x \in \{l, a, b\}, k \in \text{dom}(x.\text{pos})\}, \\
\text{merge}(l, a, b).\text{cells}(k) &= \text{mvr}(\dots), \quad \text{merge}(l, a, b).\text{ranges}(k) = \text{mvr}(\dots).
\end{aligned}$$

The observation is computed as in Section 2 with $\text{live}(\alpha, id) \Leftrightarrow id \in \text{known}(\alpha) \wedge \text{tok}(\alpha, id) \neq \emptyset$ and the register in place of the latest candidate.

The design has no cross-component rule. The reference's update-wins policy (a concurrent write or move defeats a removal) is already token-based: the write's token is not in the ancestor, so mvr keeps it. Every component is an observed-remove set or a last-writer-wins register.

4 H1: differential campaign

Conjecture 4.1 (H1). *For every legal execution of the version-DAG semantics with the canonical virtual merge base, the materialised design with named removal has the same observation as the union model at every version.*

Method. The harness (`model.py`) runs the Python reference and each materialised variant in lockstep on random version DAGs. Operations are generated from the reference state so that before-images, overwrite lists, and killed-token lists are those the issuance predicate demands; identifiers are fresh by construction and timestamps come from per-replica Lamport clocks, so a stale replica can issue events below another replica's cutoff. Replica 0 issues purge markers under

the Lean rule `purgeApplicable`, with roster acknowledgements from frontier evidence. Two to four replicas perform one to three operations each per round for three to eight rounds; a merge between two heads occurs with probability 0.7 per round; with probability 0.5 one laggard replica never merges until the forced convergence rounds, which then merge all heads along different topologies. When a recorded version’s event set equals the heads’ intersection it is the ancestor; otherwise the ancestor is the fold of the intersection in timestamp order, which is the canonical virtual merge base. At every version the harness compares observations, compares observations of versions reached by different topologies over the same event set, and compares the merge result with the timestamp-order replay of the union. Every generated event, after erasing *kills*, is checked against a transcription of `applicableB` and `clockedB`; all 90,859 events of the three seeds pass, so the executions are honest in the Lean sense and not only in the generator’s. The generator applies decision D1 (Section 6): an inverse cell write is issued only while its axes are live.

Result. Table 2 reports three seeds of 1000 executions; an execution counts against every design that fails in it. The named-removal design with purge masks has no failing execution. The clearing variant matches the reference and converges but fails the replay comparison: a removal that clears all tokens and a concurrent keep do not commute, so replay order matters and a Lean proof would need the arbitration arm of the replay policy. The named variant makes every *concurrent* pair commute. Causally ordered pairs still do not: a keep followed by the removal that names its token leaves the token absent, the reverse order leaves it present, and the same holds for an overwrite and the version it names. So universal commutation fails and the all-commuting proof route is unavailable; Section 10 gives the conditioned route instead.

Design	seed 1	seed 2	seed 3	first failure class
named removal, masks, registers kept	0	0	0	
same, retire dead tokens and <code>known</code> : descendant evidence	0	0	0	
same, retire: frontier evidence	0	0	0	
same, retire: immediately when dead	0	0	0	
named removal, purge without mask	6	3	3	masked concurrent write shown
named removal, drop unreferenced dead registers	38	41	35	position of revived axis
clearing removal	227	224	224	merge $\neq$ replay
ancestor ignored (control)	520	511	512	rows, cells, ranges
eager range re-anchoring (control)	747	720	746	ranges

Table 2: Failing executions out of 1000 per seed under the D1 issuance rule with Lamport clocks, laggard replicas, and purge markers. Each seed produced about 36,800 versions, 30,300 operations, 6,400 merges at a registered ancestor, 150 merges at a virtual base, 28 to 41 purge markers, and 6 to 12 cell writes concurrent with a marker and below its cutoff.

5 H2: the last position of a dead axis is load-bearing

Each witness is hand-derived from the definitions of Section 2 and checked by evaluating the Python reference; the checks are the functions `h2_witnesses` and `h2_pair3`. Identifiers: rows r_0, r_1, r_2 at positions 10, 20, 30, column c_0 .

Counterexample 5.1 (range resolution). *Let L insert r_0, r_1, r_2, c_0 and a range (r_0, r_2, c_0, c_0) . History W_1 removes r_0 . History W_2 moves r_0 to position 25 and then removes it. Both leave*

live rows $\{r_1, r_2\}$ at unchanged positions and the same range specification. The reference resolves the range to (r_1, r_2, c_0, c_0) after W_1 and to (r_2, r_2, c_0, c_0) after W_2 . A state that forgets r_0 's last position cannot give both answers.

Counterexample 5.2 (eager re-anchoring). *From the same L , branch A removes r_0 and branch B concurrently inserts r_3 at position 15. The reference resolves the merged range to (r_3, r_2, c_0, c_0) . Rewriting the range to r_0 's successor at removal time gives (r_1, r_2, c_0, c_0) . Lazy resolution over the dead position is part of the specification, not an implementation choice.*

Counterexample 5.3 (update-wins revival). *Let L insert r_0 at 10 and c_0 , and write cell (r_0, c_0) . Branch A_1 moves r_0 to 52 and then removes it; branch A_2 only removes it. Both branch states have identical live sheets: r_0 is dead. Branch B concurrently writes (r_0, c_0) . In both merges the write's token revives r_0 ; the reference places it at 52 after A_1 and at 10 after A_2 . The merge receives identical L and S_B , so if $S_{A_1} = S_{A_2}$ it cannot answer both. The information lives only on the removing branch.*

The third counterexample does not involve ranges or undo, and it survives the D1 issuance rule (35 to 41 failing executions of 1000 per seed for the register-dropping variant). The retention residue is therefore: one position register per axis identifier that has ever been known, retained until the removal is causally stable, that is, until no replica can still issue an operation concurrent with it. Under update-wins this is unavoidable; under a remove-wins policy the register could be dropped at removal. Causal stability suffices only if undo cannot revive a removed axis: under the current model (Section 6) an undo issued *after* observing the removal still revives it, and the register must then be kept as long as any replica can undo a write into the row. Decision D1 adopts the no-revival rule. The purge experiment of Section 7 then found that the register should simply be kept for every identifier ever known, one $O(1)$ entry each, while the identifier's tokens and `known` entry can be dropped as soon as it is dead.

6 Finding: undo revives a causally later removal

Finding 6.1. *In the current model, with row r_0 and column c_0 and a write at $t = 3$, removing r_0 at $t = 4$ and then undoing the $t = 3$ write at $t = 5$ is a legal history on one replica: the undo's target is observed and was issued by the same replica, and no issuance clause requires the axes to be live. The undo is a cell event and therefore a keep token for r_0 with a timestamp the removal did not observe. The row becomes live again at position 10 with an empty cell. Reproduced by `undo_revival_witness`.*

Table 4 of the paper covers undo against a *concurrent* removal; it does not say what undoing an edit should do to a row removed causally later. The behaviour is a consequence of representing undo as a cell event. If it is unintended, the fix is an issuance clause (undo of a cell write requires live axes) or a non-keeping inverse; either changes the public sequential specification.

Decision D1. No revival, as an issuance rule: an inverse cell write is legal only while its row and column are live. The legacy revival behaviour stays documented by this finding and its SPOT; the equivalence theorem of Section 10 is stated only for honest executions under the new rule.

7 H3: purge and retirement

The harness generates purge markers under the current model’s issuance rule and transcribes the marker semantics of `AegisSheetGC.lean`: a marker carries a cutoff, a set of dead coordinates, the covered cell versions, and roster acknowledgements; it masks every cell version at those coordinates with timestamp at or below the cutoff, including versions that arrive later from a stale branch. The Python reference reproduces the five marker fixtures of that file (the marker is issuable after the removal, a restored row shows no purged content, a post-cutoff write is issuable and visible).

Strict deletion is refuted. A materialised purge that deletes the covered versions and stores nothing diverges from the reference in 6, 3, and 3 executions per 1000. Every case has the same shape: a replica that has not observed the marker writes into a purged coordinate with a Lamport timestamp at or below the cutoff; the reference masks the write permanently, the strict design keeps its version, and the write’s keep token revives the axis so the difference is visible. This is the current model’s chosen semantics for a late concurrent edit, and the `mvr` rule alone cannot express it because the write is absent from the ancestor.

A mask suffices. Keeping, per marker, the pair (cutoff, coordinates) and filtering versions through the accumulated masks at apply and at merge reproduces the reference with no failing execution. The mask is the irreducible purge residue. The marker’s covered entries are unnecessary because tokens live in their own component, and its acknowledgements are unnecessary for the merge.

Retirement needs no acknowledgements. Three retirement rules for a dead identifier were tested: immediately when dead; when every roster member has authored any event (frontier evidence, the rule `frontierAcknowledgements` encodes); and when every roster member has authored the removal or an event whose causal past contains it (descendant evidence). All three drop the identifier’s token set and `known` entry and keep its position register, and all three have no failing execution. The reason is structural: a stale branch on which the identifier is still live carries its own `known` entry and tokens, and the merge unions `known` and applies `mvr` to tokens, so a late concurrent keep revives the identifier correctly whatever the other branch forgot. Dropping the register instead fails (Table 2), and dropping it only when no live range names it fails too, because an undone range edit can name the identifier again and the decision would then depend on the order in which references appeared rather than on the event set.

D1 is load-bearing for retirement. Under the legacy undo rule (`--legacy-undo`, seed 7) immediate retirement fails 53 of 1000 and descendant retirement 2 of 1000: an undone cell write revives an identifier whose `known` entry is gone, so the reference shows it live and the materialised state does not. The no-retirement baseline stays clean under both rules.

Consequence. The roster acknowledgement protocol of the current model plays no role in the materialised design’s convergence, masking, or retirement. It remains an issuance policy on when a purge may be requested. One decision is open, D2: whether a write concurrent with a purge and below its cutoff should be masked, as the current model does, or kept, as the maskless design does; the harness follows the current model.

8 Measured: state size

Rounds	Operations	Reference (timestamps stored)	Materialised, named, keep
4	23	122	30
8	44	469	53
16	88	2,154	101
32	176	10,076	192

Table 3: Mean over 20 executions with 3 replicas, seed 1, at the final version of replica 0. The reference count is one per event plus its `seen` set, overwrite list, and marker payload; the materialised count is tokens, registers, versions, mask entries, and known identifiers.

Across the measured range, doubling the history roughly quadruples the reference count and doubles the materialised count. This is a timestamp-count measurement consistent with quadratic versus linear scaling, not an asymptotic result. The gap at 176 operations is a factor of 53. The counts are taken in the Python models, not bytes of any runtime.

9 Controls

Positive control. The Python reference reproduces twenty hand-derived expectations copied from the Lean SPOTs, five issuance fixtures (honest and dishonest edits, own and remote undo, undo of insert), and five purge fixtures from `AegisSheetGC.lean`: update defeats concurrent removal, later removal wins, concurrent writes keep both values, selective undo keeps the remote value, move defeats removal, later move wins, insert survives adjacent removal, undo of remove restores the position, undo of insert removes only the inserted identifier, the two Table 4 move-undo cases, and seven Figure 1 range cases (`check_fixtures`).

Negative controls. Two deliberately wrong designs fail on every seed (Table 2): ignoring the ancestor, which reintroduces removed versions, and eager range re-anchoring. A harness that could not distinguish these would carry no evidence.

10 Mechanization correspondence

Display	Lean declaration	Status
Reference events, state, liveness, positions, cells, ranges, resolution, issuance	<code>Instances.AegisSheet: D, axisLive, axisPositions, cellValues, rangeValues, resolveRange, applicableB</code>	machine-checked definitions; transcription validated on 20 fixtures
Materialised state, update, merge, purge mask, retirement	none	specified; Python implementation
Conjecture H1	none	validated on the campaign scope
Counterexamples 5.1 to 5.3, Finding 6.1	none	hand-derived, checked by the Python reference; to be pinned as Lean SPOTs

Lean plan. The materialised design is a product of observed-remove sets and one last-writer-wins register. Because causally ordered pairs do not commute, the all-commuting route is closed. The token and version components need `JoinAt` under an honesty predicate on the replay context (every removal’s *kills* and every overwrite’s *overwrites* lie in the issuer’s causal past), discharged from issuance with `IssuanceEstablishes` as `Queue` and `EmbedRGA` do, and composed with `joinAt_prod`; the register alone is all-commuting. A `VerifiedMRDT` for the new package proves correctness against its own sequential specification only. H1 itself needs a separate theorem: an erasure dropping *kills* that maps honest executions of the new package to honest executions of the union model, a representation relation between materialised states and event sets preserved by `do` and `merge`, and equality of observations under it. The sequential certificate needs a new representation relation; `no_view_only_step` shows the visible sheet is insufficient, and `row_tokens_distinguish_future`, `cell_versions_distinguish_future`, and `range_versions_distinguish_future` fix that tokens, cell version identities, and range version identities must be retained, which the materialised state does.

11 Open obligations

1. Decision D1 (no revival) is taken; carry it into the Lean issuance predicate of the new package and keep the legacy SPOT.
2. Decide D2 (Section 7): mask or keep a write concurrent with a purge and below its cutoff. The Lean port carries the mask either way if the current model’s semantics is kept.
3. Pin Counterexamples 5.1 to 5.3 and Finding 6.1 as Lean SPOTs against the existing model, with PASS and FAIL companions.
4. Port the named-removal design to Lean as a second package beside the current one; prove `JoinOn` under the honesty predicate, the sequential certificate, and the cross-model observational-equivalence theorem; keep the union model until the runtime measurement is in.
5. Extend the harness to merges between non-head versions and to undo of undo; report the generator’s coverage.